Università degli
Studi di Milano-Bicocca

Summing up + Pytorch Lightning

Prof. Flavio Piccoli - Dr. Mirko Paolo Barbato

R&D process

Analysis of the state of art

↓

Data collection / analysis

↓

Architectural Design

↓

Parameter Search

↓

Benchmarking

↓

Deployment

pandas

scikit learn

Data preprocessing

Cleaning

Deal with missing data

Encoding

Convert data in a machine-learning suitable format

Normalizing

Normalize the variables

Splitting

Split in train / validation / test

Cleaning

What do we do with missing data?

Three possible strategies:

- discard feature having missing data
- discard samples having missing data
- substitute missing data with plausible content
    - **booleans / categorical**: replace with mode
    - **integers**: replace with median
    - **floats**: interpolate

Encoding of input categorical variables

- Machine learning models can only work with numerical values
- It is necessary to transform the categorical values of the relevant features into numerical ones

One-hot encoding (for input variables)

brand
Fiat
BMW
Lamborghini
Fiat

→

brand_fiat	brand_bmw	brand_lambo
1	0	0
0	1	0
0	0	1
1	0	0

Encoding of target categorical variables

- Machine learning models can only work with numerical values
- It is necessary to transform the categorical values of the relevant features into numerical ones

Labeling (for estimated variables)

brand
Fiat
BMW
Lamborghini
Fiat

→

brand
0
1
2
0

Normalizing

- The range of the variables affect their importance
- We need to normalize them so that each variable resides in the same range

  - **min - max normalization**
    - if the variable under analysis has a specific range, it's possible to use this normalization
    - `from sklearn.preprocessing import MinMaxScaler`

  - **standardization**
    - if the range is unknown a priori
    - sets the mean to 0 and the variance to 1
    - `sklearn.preprocessing import StandardScaler`

Splitting

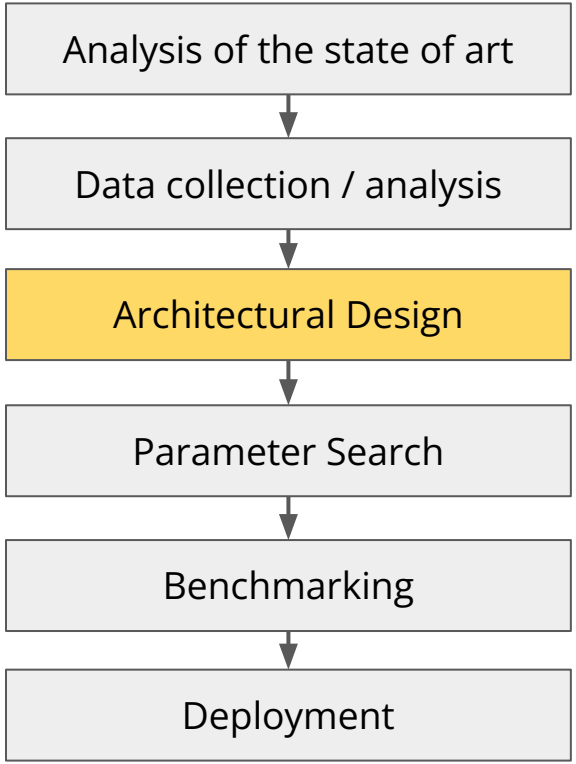
It is possible to split in train, validation and test with the following code:

```
# define percentage of splitting
train_perc = 0.8
val_perc = 0.1
# test_perc will be: 1 - train_perc - val_perc

# split train validation and test
train = df.sample(frac = train_perc, random_state=1)
test = df.drop(train.index).sample(frac = val_perc/(1-train_perc), random_state=1)
val = df.drop(train.index).drop(test.index)
```

$$(1-0.8)x = 0.1 \qquad x = \frac{0.1}{1-0.8}$$

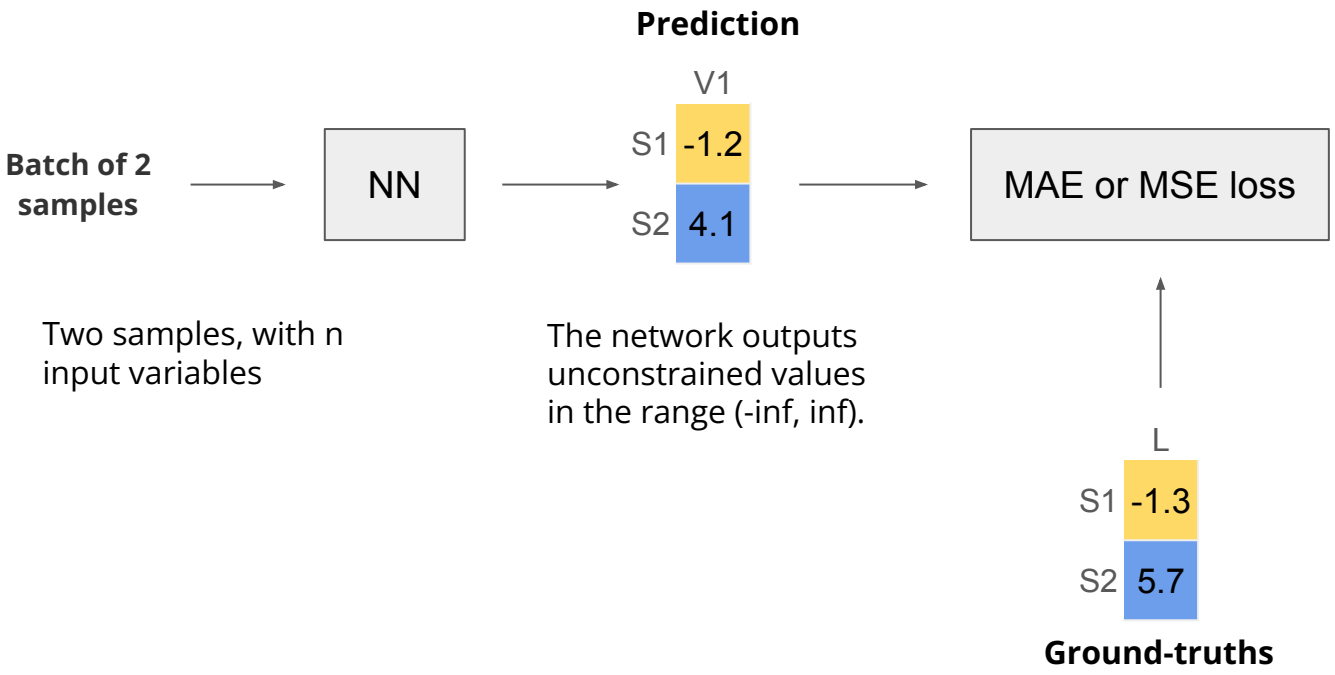
R&D process



PyTorch PyTorch Lightning

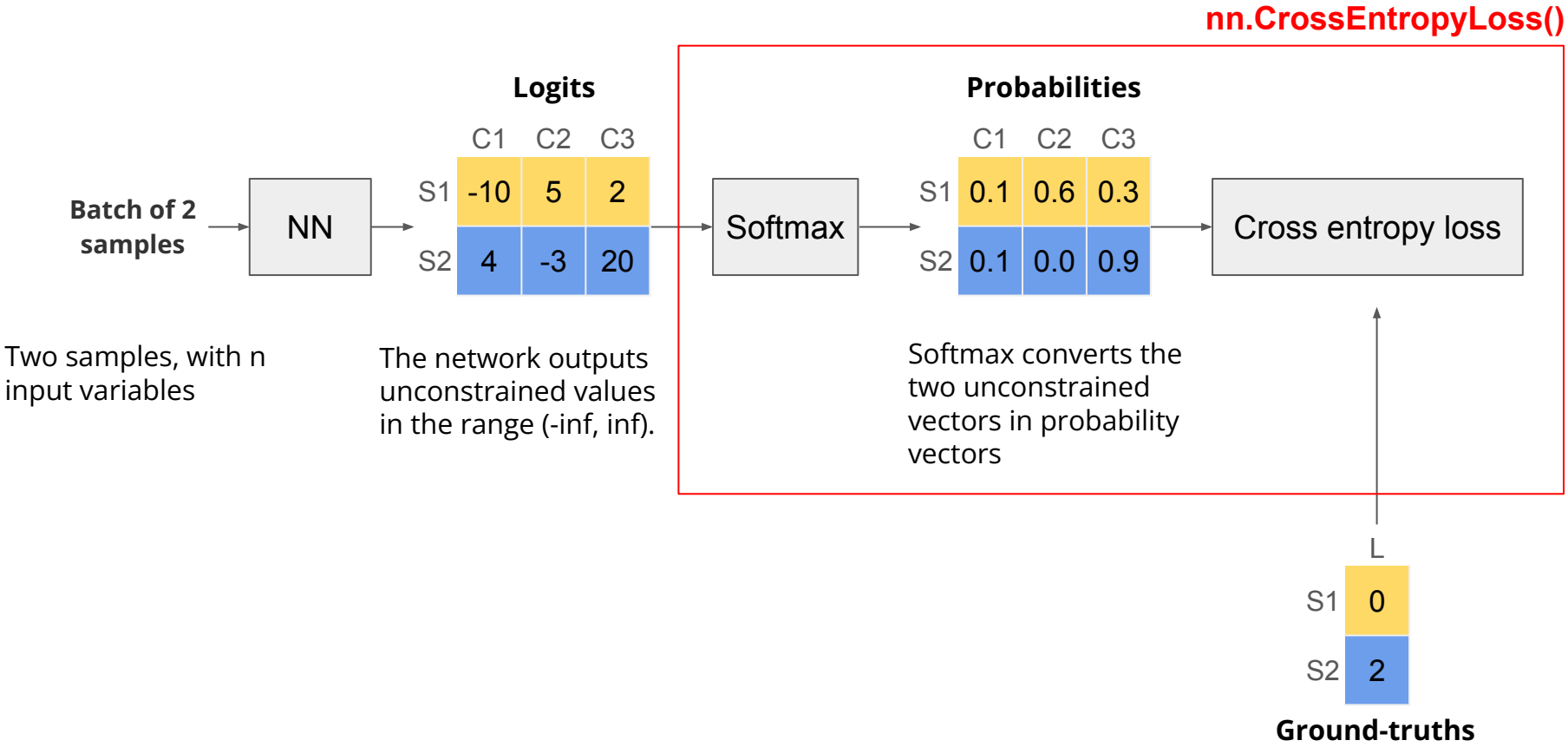
Regression setup

- network predicts directly the values of the continuous variable
- loss and performance score are: MAE or MSE



Pipeline for classification in Pytorch

In Pytorch, nn.CrossEntropyLoss combines softmax and cross entropy loss



Accuracy

Micro

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Macro

$$\text{Accuracy} = \frac{1}{N}\sum_{c=1}^{N}\frac{\text{\# correct preds for class c}}{\text{\# samples of class c}}$$

Pred	GT
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	1
0	1

$Acc = 0.8$

$Acc_0 = 1$

$Acc_1 = 0$

$Acc = 0.5$

Torchmetrics

```python
import torchmetrics

# define input and ground truth
inp = torch.tensor([0,0,0,0,0,0,0,0,0,0])
gt  = torch.tensor([0,0,0,0,0,0,0,0,1,1])

# define metric objects
acc_micro = torchmetrics.Accuracy(task = 'multiclass', num_classes = 2, average = 'micro')
acc_macro = torchmetrics.Accuracy(task = 'multiclass', num_classes = 2, average = 'macro')

# update metrics
acc_micro.update(inp, gt)
acc_macro.update(inp, gt)

# you can update the metrics with more batches ..

# at the end, compute the final score
micro = acc_micro.compute()
macro = acc_macro.compute()

# print
print(f'Micro accuracy is {micro:0.2f} while macro accuracy is {macro:0.2f}')

# reset the metric object (optional)
acc_micro.reset()
acc_macro.reset()
```

Initialization

Update of the metric.
One update for each batch.

Final computation of the metric

Reset of the metric

It will print: “Micro accuracy is 0.80 while macro accuracy is 0.50”

Pytorch Lightning

Pytorch Lightning

PyTorch Lightning is a powerful framework that helps **standardize and streamline deep learning code**, reducing boilerplate and improving readability. It enforces a structured approach by organizing key components into well-defined functions:

- **Initialization (`__init__`)** – Define the **model** architecture, optimizer, and other hyperparameters.
- **Training Step (`training_step`)** – Implement the logic for **forward propagation** and loss computation during training.
- **Validation Step (`validation_step`)** – Evaluate model **performance** on a validation set.
- **Test Step (`test_step`)** – Assess final model performance on unseen test data.

There are also several optional functions that you can extend:

- **Validation/Test Start (`on_validation_start` / `on_test_start`)** – Called once before the validation/test loop begins (e.g., for setup, logging, or resetting metrics).
- **Validation/Test End (`on_validation_end` / `on_test_end`)** – Called once after the validation/test loop finishes (e.g., for aggregating results or logging summaries).

By following this modular structure, PyTorch Lightning makes research more reproducible, training more scalable, and code easier to debug and maintain.

__init__

PyTorch Lightning

- You will have to extend pl.LightningModule
- In the init, you will define the network and the loss
- Optionally, you will specify the (torch)metrics that you will use to measure your model

```python
class Experiment(pl.LightningModule):
    def __init__(self, conf):
        super().__init__()
        # save parameters
        self.conf = conf
        self.save_hyperparameters(conf)
        # define network
        self.net = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
        self.net.fc = nn.Linear(self.net.fc.in_features, self.conf['num_classes'])
        # define loss
        self.loss_fun = nn.CrossEntropyLoss()
        # Define torchmetrics in a collection
        self.metrics = {}
        self.metrics['micro'] = torchmetrics.Accuracy(task='multiclass', num_classes=self.conf['num_classes'], average='micro')
        self.metrics['macro'] = torchmetrics.Accuracy(task='multiclass', num_classes=self.conf['num_classes'], average='macro')
        self.metrics = torchmetrics.MetricCollection(self.metrics)

```

train_step

PyTorch Lightning

- You must specify how to process a training batch
- You can log your loss(es) value(s)
- You must return a value. It will be used for backpropagation.
  - If you have more than one loss, average them and return a single value

```python
def training_step(self, batch, batch_idx):
    # split input and target
    src, tgt = batch
    # compute output
    out = self.net(src)
    # compute loss
    loss = self.loss_fun(out, tgt)
    # log
    self.log("train/loss", loss, prog_bar=True)
    return loss
```

validation

- on_validation_start and on_validation_epoch_end are optional, but useful to compute the metrics over the entire validation set

```python
    def on_validation_start(self):
        # reset metrics at the beginning of validation
        self.metrics.reset()


    def validation_step(self, batch, batch_idx):
        # perform prediction
        src, tgt = batch
        out = self(src)
        # update metrics
        self.metrics.update(out, tgt)


    def on_validation_epoch_end(self):
        # compute metrics
        results = self.metrics.compute()
        # log them
        for k, v in results.items():
            self.log(f"val/{k}", v, prog_bar=True)
```

test

- same thing for the test

```python
    def on_test_start(self):
        self.metrics.reset()

    def test_step(self, batch, batch_idx):
        src, tgt = batch
        out = self(src)
        self.metrics.update(out, tgt)

    def on_test_epoch_end(self):
        results = self.metrics.compute()
        for k, v in results.items():
            self.log(f"test/{k}", v, prog_bar=True)
```

optimizer & data loading

- in configure_optimizers, you will pass the weights that you want to optimize. In this case, all network's parameters
- *train_dataloader*, *val_dataloader* and *test_dataloader* must return a pytorch dataloader
- each dataloader will take as input
  - your dataset class (same as before)
  - the batch size
  - shuffle (true or false if you want to shuffle the instances at the beginning of each epoch). Usually True only in training
  - drop_last if you want to discard last batch if it does not have enough samples to fill batch_size. Usually True only in train

```python
    # ------------------ OPTIMIZER ------------------

    def configure_optimizers(self):
        return optim.Adam(self.net.parameters(), lr=self.conf['learning_rate'], weight_decay=self.conf['weight_decay'])

    # ------------------ DATA ------------------

    def train_dataloader(self):
        return data.DataLoader(Dataset(self.conf, 'train'), batch_size=self.conf['batch_size'], shuffle=True, num_workers=self.conf['num_workers'])

    def val_dataloader(self):
        return data.DataLoader(Dataset(self.conf, 'val'), batch_size=self.conf['batch_size'], shuffle=False, num_workers=self.conf['num_workers'])

    def test_dataloader(self):
        return data.DataLoader(Dataset(self.conf, 'test'), batch_size=self.conf['batch_size'], shuffle=False, num_workers=self.conf['num_workers'])
```

training function

- read the json file with all the parameters
- define the callbacks to save the models
- train

```python
def train(args):
    # Define output directory
    outdir = os.path.splitext(os.path.basename(args.train))[0]
    os.makedirs(outdir, exist_ok=True)

    # read the json file with all the arguments
    with open(args.train) as f:
        conf = json.load(f)
    model = Experiment(conf)

    # Define callbacks
    callbacks = []
    # add a callback for micro and macro accuracy
    callbacks.append(pl.callbacks.ModelCheckpoint(monitor=f'acc/micro', mode='max', save_last=True, filename='micro'))
    callbacks.append(pl.callbacks.ModelCheckpoint(monitor=f'acc/macro', mode='max', save_last=True, filename='macro'))
    # Define tensorboard logger
    logger = pl.loggers.TensorBoardLogger(outdir)
    # define trainer
    trainer = pl.Trainer(max_epochs=args.epochs, callbacks=callbacks, logger=logger)
    # launch training
    trainer.fit(model)
```

test function

- load the checkpoint with CLASS.load_from_checkpoint(string path)
- define a trainer object, which will be used for testing
- return the computed values

```python
def test_model(args):
    # load model
    model = Experiment.load_from_checkpoint(args.checkpoint)
    # test
    trainer = pl.Trainer()
    return trainer.test(model)
```

The logging mechanism

How to log a quantity

- In any function, you can log a quantity with self.log
- You need to associate a name to the quantity, e.g. “train/loss”
- if prog_bar is true, you will see the quantity in the progress bar

```python
    def training_step(self, batch, batch_idx):
        # split input and target
        src, tgt = batch
        # compute output
        out = self.net(src)
        # compute loss
        loss = self.loss_fun(out, tgt)
        # log
        self.log("train/loss", loss, prog_bar=True)
        return loss
```

How to use the quantity

- You can later access to the quantity with callbacks
- Just specify the quantity name in the monitor parameter
- Other parameters:
  - mode: min if the lower the better, otherwise max
  - save_last: if you also want to save the last model

```python
def train(args):
    # Define output directory
    outdir = os.path.splitext(os.path.basename(args.train))[0]
    os.makedirs(outdir, exist_ok=True)

    # read the json file with all the arguments
    with open(args.train) as f:
        conf = json.load(f)
    model = Experiment(conf)

    # Define callbacks
    callbacks = [
        pl.callbacks.ModelCheckpoint(monitor=f'train/loss',
                                     mode='min',
                                     save_last=True,
                                     filename='loss')
    ]
    # Define tensorboard logger
    logger = pl.loggers.TensorBoardLogger(outdir)
    # define trainer
    trainer = pl.Trainer(max_epochs=args.epochs,
                         callbacks=callbacks,
                         logger=logger)
    # launch training
    trainer.fit(model)
```

The logging mechanism

How to log a quantity

- In any function, you can log a quantity with self.log
- You need to associate a name to the quantity, e.g. “train/loss”
- if prog_bar is true, you will see the quantity in the progress bar

```python
    def training_step(self, batch, batch_idx):
        # split input and target
        src, tgt = batch
        # compute output
        out = self.net(src)
        # compute loss
        loss = self.loss_fun(out, tgt)
        # log
        self.log("train/loss", loss, prog_bar=True)
        return loss
```

How to use the quantity

- You can later access to the quantity with callbacks
- Just specify the quantity name in the monitor parameter
- Other parameters:
  - mode: min if the lower the better, otherwise max
  - save_last: if you also want to save the last model

```python
def train(args):
    # Define output directory
    outdir = os.path.splitext(os.path.basename(args.train))[0]
    os.makedirs(outdir, exist_ok=True)

    # read the json file with all the arguments
    with open(args.train) as f:
        conf = json.load(f)
    model = Experiment(conf)

    # Define callbacks
    callbacks = [
        pl.callbacks.ModelCheckpoint(monitor=f'train/loss',
                                     mode='min',
                                     save_last=True,
                                     filename='loss')
    ]
    # Define tensorboard logger
    logger = pl.loggers.TensorBoardLogger(outdir)
    # define trainer
    trainer = pl.Trainer(max_epochs=args.epochs,
                         callbacks=callbacks,
                         logger=logger)
    # launch training
    trainer.fit(model)
```

Tensorboard

- You can use the tensorboard as well by passing it in the logger argument of the trainer
- You can use also other tools such as Weights & Biases (wandb) for logging and storing parameters online

```python
def train(args):
    # Define output directory
    outdir = os.path.splitext(os.path.basename(args.train))[0]
    os.makedirs(outdir, exist_ok=True)

    # read the json file with all the arguments
    with open(args.train) as f:
        conf = json.load(f)
    model = Experiment(conf)

    # Define callbacks
    callbacks = [
        pl.callbacks.ModelCheckpoint(monitor=f'train/loss',
                                     mode='min',
                                     save_last=True,
                                     filename='loss')
    ]
    # Define tensorboard logger
    logger = pl.loggers.TensorBoardLogger(outdir)
    # define trainer
    trainer = pl.Trainer(max_epochs=args.epochs,
                         callbacks=callbacks,
                         logger=logger)
    # launch training
    trainer.fit(model)
```

Loading a trained model

To load a model is very easy.

- You can use CLASS_NAME.**load_from_checkpoint**
- You can either pass just the filename of the checkpoint
- Or you can pass as many variables as you want to change their value

```
# define filename
checkpoint_path = 'lightning_logs/version_0/checkpoints/macro.ckpt'

# load model (all parameters are loaded just like in the training)
model = Experiment.load_from_checkpoint(checkpoint_path)

# load model changing the value of my_var
model = Experiment.load_from_checkpoint(checkpoint_path, my_var=10)

# load model changing the batch size in the configuration (the dictionary is updated)
model = Experiment.load_from_checkpoint(checkpoint_path, conf={'batch_size': 10})
```

Exercise

Exercise 1 - Pytorch Lightning

- Take the classification exercise of the last lesson and convert the code in Pytorch Lightning format

Exercise 2 (here or at home) - Data analysis + neural prediction

- Given the dataset “traffic_violations” with target variable “is_arrested”:

1. explore the data
2. decide which variables should be dropped (e.g. 'country_name', 'stop_date', 'stop_time')
3. clean data
4. encode data
5. normalize data
6. split data in train, validation and test using the code in slide 8
7. set up the training of a neural network
8. train the system
9. test the performance in terms of micro and macro accuracy
10. compute also the confusion matrix